```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
```

```
from google.colab import files

uploaded = files.upload()

df = pd.read_excel("student_grade_ses_dataset.xlsx")

df
```

```
x = df.drop(columns=[
    "student_id",
    "survey_date",
    "final_grade_pct",
    "grade_class",
    "at_risk_flag"
])

y = df["grade_class"]

print("Features names", list(x.columns))
print("Target names", y.unique())
print("Number of samples", x.shape[0])
print("Number of features", x.shape[1])
```

```
Features names ['academic_year', 'age', 'gender', 'grade_level', 'school_
Target names ['D' 'C' 'B' 'A' 'F']
Number of samples 12025
Number of features 24
```

```
df1 = x.copy()

df1["grade_class"] = y

df1.head()
```

```
print(df1["grade_class"].value_counts())
```

```
grade_class
C    6161
D    2744
B    2689
F     219
A     212
Name: count, dtype: int64
```

```
print("Shape of x", x.shape)
print("Shape of y", y.shape)
```

```
Shape of x (12025, 24)
Shape of y (12025,)
```

```
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=0,
    stratify=y
)

print("Shape of x_train", x_train.shape)
print("Shape of x_test", x_test.shape)
```

```
Shape of x_train (9620, 24)
Shape of x_test (2405, 24)
```

```
numeric_features = x.select_dtypes(
    include=np.number
```

```
).columns

categorical_features = x.select_dtypes(
    include="object"
).columns

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), numeric_features),

    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_features)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

model.fit(x_train, y_train)

print("Model completed")
```

```
Model completed
```

```
y_pred = model.predict(x_test)

results = pd.DataFrame({
    "actual": y_test.values,
    "predicted": y_pred
})

results.head(10)
```

Next steps: Generate code with `results` New interactive sheet

```
accuracy = accuracy_score(y_test, y_pred)

print(f"Test accuracy: {accuracy:.2%}")
```

```
Test accuracy: 58.46%
```